Problema 2

1. **Nomes/identificadores e Variáveis:**
    a. Os nomes identificam um endereço na memória de uma variável, permitindo uma melhor legibilidade em relação à qual unidade de memória está sendo referida.
    b. Os seis atributos que permitem caracterizar uma variável são:
        i. Tipo: o tipo determina o tamanho da memória que a variável vai ocupar e qual o domínio de valores ela pode assumir.
        ii. Escopo: a delimitação de sentenças na qual a variável é visível.
        iii. Tempo de vida: a duração entre a alocação e a desalocação da variável na memória.
        iv. Endereço: a posição inicial da memória de uma variável, determinando onde começa a unidade de endereçamento.
        v. Nome: o identificador legível da variável.
        vi. Valor: a informação contida na memória que a variável armazena.
    c. Aliases podem ser criados criando uma referência à um mesmo endereçamento na memória com variáveis diferentes, por exemplo com dois ponteiros apontando para o mesmo endereço.
    d. As opções serão variáveis de pilha e variáveis de heap explícitas, e variáveis estáticas constantes.
2. **Bindings em Linguagens de Programação:**
    a. Binding pode ser definido como a associação entre um atributo e uma entidade, como um nome e o tipo que ele representa, ou uma variável e o valor que ela assume. Podemos distinguir, entre outros, o binding de tipo, o binding de armazenamento/endereço, o binding de valor e o binding de escopo de uma variável.
    b. Binding time é o momento em que uma associação é estabelecida. Pode ocorrer em tempo de projeto da linguagem, de implementação, de compilação, de carregamento (load time) ou de execução. Bindings

fixados antes da execução e que não mudam são estáticos; os que ocorrem ou podem mudar durante a execução são dinâmicos.

c. Em tempo de compilação ocorrerão o binding de tipo de cada variável (tipagem estática), a resolução de escopo, o binding de armazenamento das variáveis de pilha e a verificação de imutabilidade, garantindo que uma variável imutável não seja reatribuída.
d. Em tempo de execução ocorrerão o binding de valor, ou seja, a atribuição efetiva de um valor a uma variável quando a instrução é executada, e o binding de armazenamento das variáveis de heap explícitas, cujo endereço só existe a partir da alocação efetivamente executada pelo programa.

3. **Escopo e Tempo de Vida:**
    a. Escopo é a faixa de sentenças na qual uma variável pode ser referenciada ou atribuída, e tempo de vida é a faixa de tempo entre a variável ser alocada na memória e a variável ser desalocada.
    b. São conceitos relacionados, o tempo de vida da variável geralmente inicia quando seu escopo inicia e termina quando sai de escopo.
    c. O escopo é um conceito textual e o tempo de vida é um conceito temporal. Além disso, o tempo de vida de uma variável pode diferir do seu escopo. Por exemplo, se eu declarar uma variável em um método e esse método, depois, chamar outro, o tempo de vida da variável vai se estender ao segundo método, mas o escopo dela se restringe apenas ao método original.
4. **Resolução de Escopo, Ambiente de Referenciamento e Blocos:**

    a.) Resolução de escopo

    Duas abordagens são discutidas na literatura: escopo estático (léxico), em que a vinculação de um nome à variável referenciada é determinada pela estrutura textual do programa e pode ser resolvida inteiramente em tempo de compilação; e escopo dinâmico, em que essa vinculação depende da sequência de chamadas de subprogramas em tempo de execução, prejudicando a legibilidade, já que o significado de uma referência passa a depender do contexto de chamada, e não apenas da leitura do texto. Coerente com a prioridade

em legibilidade e confiabilidade já assumida no Subproblema 1, a linguagem adota exclusivamente resolução de escopo estática, resolvida integralmente durante a compilação ahead-of-time, sem custo em tempo de execução.

b.) Ambiente de referenciamento

É o conjunto de todos os nomes visíveis (corretamente vinculados) em um ponto do programa. Em escopo estático, corresponde à união do escopo local com os escopos de todos os blocos textualmente externos a ele, até o escopo do módulo; quando um nome é declarado em mais de um desses escopos, prevalece a declaração mais interna.

c.) Blocos

Blocos são delimitados por chaves e correspondem ao corpo de endpoints, procedimentos/funções e estruturas de controle (if, for, foreach, ramos de pattern matching), cada um introduzindo um novo escopo aninhado. Blocos aninhados são permitidos, inclusive blocos livres para restringir deliberadamente o tempo de vida de variáveis auxiliares. Para reforçar a legibilidade, a linguagem não permite shadowing implícito: redeclarar um nome já visível em escopo externo é erro de compilação, salvo uso explícito de uma palavra-chave de sombreamento intencional.

d.) Implementação na linguagem proposta

Os blocos definem a estrutura textual da qual a resolução de escopo estático deriva suas regras, e o ambiente de referenciamento de cada instrução é a composição de todos os blocos que a envolvem. O compilador implementa isso com uma tabela de símbolos em pilha: cada entrada em um bloco empilha uma nova tabela, cada saída a desempilha, e a resolução de um nome percorre a pilha do escopo

mais interno ao mais externo. Como essa resolução ocorre inteiramente em tempo de compilação, não há custo em tempo de execução, o que reforça os critérios de confiabilidade e desempenho previsível priorizados no design da linguagem.

5. **Categorias de Variáveis quanto ao Tempo de Vida e Armazenamento:**
    a. as variáveis são classificadas em quatro categorias conforme o vínculo de armazenamento e tempo de vida: estáticas, vinculadas antes da execução e que permanecem durante todo o programa; de pilha (stack-dynamic), vinculadas ao elaborar a declaração durante a execução e desvinculadas ao sair do bloco; de heap explícita, alocadas e desalocadas por instruções do programador; e de heap implícita, vinculadas por completo apenas ao receberem um valor.
    b. A linguagem terá variáveis estáticas, usadas nas constantes nomeadas; variáveis de pilha, para as variáveis locais imutáveis dentro de procedimentos e endpoints; e variáveis de heap explícitas, para List, Set, Map e valores encapsulados em Optional/Result. Não haverá heap implícita, pois sua vinculação tardia contraria a tipagem estática e a previsibilidade.
    c. O escopo das variáveis heap segue o mesmo mecanismo de escopo estático das demais: a referência é guardada em uma variável de pilha, cujo escopo é resolvido textualmente. O que a distingue é que seu tempo de vida pode ultrapassar esse escopo, como quando uma List é retornada por um endpoint e continua viva após o bloco que a criou terminar.
6. **Constantes Nomeadas:**
    a. Haverá suporte a constantes nomeadas.
    b. A vinculação de valores será feita de forma estática.
    c. A vinculação ocorrerá antes da execução do programa.